



RUTGERS-NEW BRUNSWICK
School of Engineering

Ollama: Run LLMs Locally

TA Session: Fangru Linghu

02/09/2026

Course Goals and Structure

- ❑ Text Book:
<https://drive.google.com/file/d/19dqWms9qUQ2Bi8JfsBhC5Q6zDeJd0s5o/view>
- ❑ This course will cover basic concepts of software engineering
 - Agile Methods
 - Test Driven Design
 - Software architecture with emphasis on REST and microservices
 - Testing methods
 - Advanced topics in JavaScript programming
 - Students are expected to be familiar with the basics of JavaScript and Python and we will not cover the basics of JavaScript in the class
- ❑ Goal: To build a Web Service for Agentic System (similar to ChatGPT)
 - Team projects are organized in three iterations, the details of which will be specified
- ❑ All students are required to use only the fundamental web technologies: JavaScript, HTML, CSS, and Python
 - As for the libraries or frameworks, nothing other than NodeJS to run JavaScript on the server side, an SQL-based database (example: MySQL) or NoSQL database (example: Firebase), vLLM as the LLM serving platform.

Course Project

- ❑ Each team has 6 students
- ❑ The goal is to build a web service for an agentic system
 - You can think about it as ChatGPT or Gemini
 - There is a large language model (LLM) in the backend
 - There is a web user interface in the frontend
 - Users input prompts and get generated texts from LLM
- ❑ The project consists three iterations
 - Iteration 1: Authentication, Feb 26 – Mar 16
 - Iteration 2: TBD, Mar 17 – Apr 9
 - Iteration 3: TBD, Apr 10 – Apr 30
- ❑ There are three roles for each iteration, students will rotate their roles across iterations
 - Requirement Engineers: Requirement analysis
 - Test Engineers: Test Design and Acceptance
 - Implementation Engineers: Development, Unit Tests

What is Ollama

- Ollama = local LLM runtime + model manager
- Lets you download (“pull”), run, and serve LLMs on your machine
- Provides:
 - **CLI** for interactive use
 - **Local HTTP API** for programmatic calls (your backend can call it)
 - <http://127.0.0.1:11434> by default

Why Ollama

- There are multiple tools for running LLMs locally:
- vLLM
 - High-performance GPU inference
 - Optimized for high concurrency and throughput
 - Commonly used when deploying LLMs as a scalable service
- llama.cpp
 - Lightweight C/C++ inference engine
 - Efficient on CPU and low-resource machines

Why Ollama

- Ollama
 - Simple installation across macOS, Windows, and Linux
 - Unified interface
 - Both CLI and Local HTTP API
 - Easy model management (pull, run, list)
- Ollama is the best match of our course goal: Focus on system design, not deployment complexity or inference optimization

Demo

Installation

- Step1: Install Ollama from the official website
 - <https://docs.ollama.com/quickstart>
- Step2: Open your Terminal:

```
# Start ollama  
ollama serve
```

```
# Check intallation  
ollama --version
```

```
# Help  
ollama --help
```


Model Selection

- Smaller models
 - Faster, lower memory usage
 - Good for basic chat, simple reasoning, quick demos
- Larger models
 - Better reasoning and generation quality
 - Slower and more memory-intensive

Model Selection

Total memory $\approx$ Model weights + KV cache + Runtime overhead

≤ 8 GB RAM

→ Use small models (e.g., $\sim 2\text{B}$ – 4B parameters)

16 GB RAM

→ Mid-size models ($\sim 7\text{B}$ parameters) usually work well

32 GB+ RAM or GPU available

→ Larger models are possible, but expect higher latency

Model Selection

- Find supported models
 - <https://ollama.com/library>

Model Selection

- Frequently Used Commands:

```
# List all locally available models  
ollama list
```

```
# Inspect model details and configuration  
ollama show llama3.2
```

```
# Run a model (first run will download it automatically)  
ollama run deepseek-r1:14b
```

```
# Pull a model in advance  
ollama pull qwen2.5:7b
```

Customize Your Model

- Ollama provide **Modelfile**, letting you define default behavior for a model.
- Example:

```
### Modelfile
FROM deepseek-r1:14b
# sets the temperature to 0.2 [higher is more creative, lower is
more coherent]
PARAMETER temperature 0.2
# sets the context window size to 4096, this controls how many
tokens the LLM can use as context to generate the next token
PARAMETER num_ctx 4096
```

Customize Your Model

1. Save it as a file (e.g. `Modelfile`)
2. `ollama create choose-a-model-name -f <location of the file`
`e.g. ./Modelfile>`
3. `ollama run choose-a-model-name`

```
# View the Modelfile of a givin model
ollama show --modelfile llama3.2
# Verify the usage
ollama ps
```

Customize Your Model

Trade-offs:

- Context length
 - Longer context allows the model to take in more information
 - Comes with higher memory usage due to a larger KV cache
 - Can significantly increase latency, even for short inputs
 - Very long contexts may also lead to context rot, where earlier information becomes less effective
- Model size
 - Larger models may provide better reasoning and generation quality (maybe)
 - Require more RAM / VRAM to load model weights
 - Smaller models are often fast, stable, and sufficient for many simple tasks

Customize Your Model

Trade-offs:

- Practical takeaway
 - Context length and model size are not “the bigger, the better”
 - They should be chosen based on task requirements and hardware constraints
 - Start from a 1~4B model with a context length of 4096, then adjust as needed

Possible Issues

- RAM/VRAM issue
 - Model may be too large for your machine
 - Symptoms: very slow response, model fails to load, or process gets killed
 - Solution: choose a smaller model or close other memory-intensive applications
- Port unavailable
 - Default port is 11434
 - Error occurs if the port is already in use
 - Solution: stop the existing Ollama service or check which process is using the port

Homework 2

In this homework, you will set up a local LLM using Ollama.

Requirements:

- 1) Install Ollama on your local machine
- 2) Select a model appropriate for your available RAM/VRAM
- 3) Run the model successfully using the Ollama CLI
- 4) Verify your setup by generating a response from the model

Submission:

1. A screenshot showing that ollama serve is running
2. A screenshot of a successful CLI test (including query and response)
3. A brief note indicating which model you used